

Programming for problem solving (Unit-3)



**Department of Computer Science and Engineering
Bhilai Institute of Technology, Durg (C.G.)**

Sudip Bhattacharya
Department of Computer Science & Engineering BIT, Durg

SYLLABUS **UNIT III: Pointers & Arrays** [CO3
1

Pointers: Defining pointers, pointer assignment, pointer arithmetic, types of pointers, Programs based on pointers.

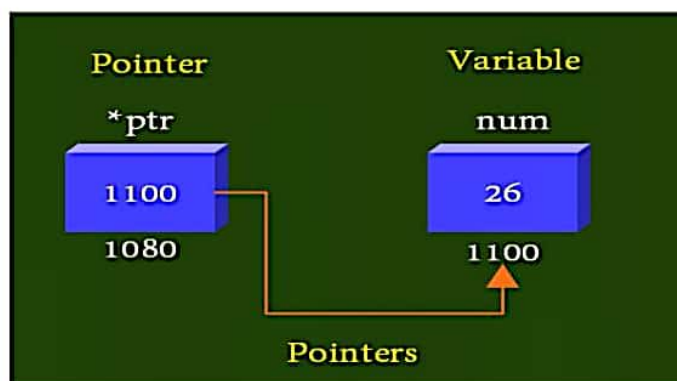
Introduction to Arrays(1-D, 2-D) : Initialization and declaration, Character arrays and Strings, Basic Algorithms: Searching (Linear Search and Binary Search), Basic Sorting Algorithms (Bubble Sort and Selection), Matrix addition, subtraction and multiplication, Accessing Arrays using Pointers, Standard string library functions (strlen, strcmp, strcmp, strcpy, strcat, etc.)

Course Outcome: CO3

- To analyzes, and interprets the concept of pointers, declarations, initialization, operations on pointers and their usage.

Pointers in C

Pointers are variables that used to store the addresses of other type of variables (or functions) in C.



In the adjacent figure,

num is an int type variable with value 26

ptr is a pointer holding the address of num (the address of num is 1100)

ptr itself is a variable (pointer type) stored at address 1080

Image source : <https://electricalworkbook.com/c-pointers/>

Addresses of variables can be obtained using the & operator (known as referencing) and can be assigned to a pointer type variable. Pointers also provide the operation of * (dereferencing) which is getting the value of the variable stored at the address contained inside a pointer variable.

& - “Address of” (Referencing) operator , used to get the address of a variable

* - “Value at” (De- referencing operator) , used to get the value at the address contained in a pointer type variable

The following code illustrates the use of the following operators using a pointer to an int type

```
// source code to store the address of int variable
#include<stdio.h>
int main()
{
    int a = 100; // int variable initialized to 100
    int * p= &a; // pointer to an integer variable
    // the pointer p holds the address of the variable address

    printf("\n Address of a is %u", &a); //address of address
    printf("\n The contents of the pointer variable p = %u", p);
    printf("\n Value at p is %d", *p);

} //end main
```

Uses of Pointers

- Passing arguments to functions using call by reference
- To Return multiple values from functions
- Passing structures and arrays to functions efficiently
- Accessing array elements using base address and offset
- Manipulating strings
- Accessing structure members
- Dynamic memory allocation
- Creating different type of data structures like lists, trees etc.
- Reading and writing directly from/into some memory location
- Low level programming, Systems programming

Pointer Arithmetic

Like other data types pointers do have some specific operations defined on them. These operations are

- Increment/Decrement
- Addition/Subtraction by integer
- Difference of two pointers
- Comparison of two pointers

Increment/Decrement of pointers

If we increment a pointer by 1, the pointer will start pointing to the immediate next location. This is somewhat different from the general arithmetic since the value of the pointer will get increased by the size of the data type to which the pointer is pointing. For e.g. in a platform with 4 byte integer , a increment of 1 to an integer pointer will result in the pointer moving 4 bytes in memory. Similarly when a character pointer is incremented by 1 , it will move 1 byte in memory. The following illustrates this concept.

```

/* Code to observe the increment in a pointer when incremented by 1 */
#include<stdio.h>
int main()
{
    int number=50;
    int *p;//pointer to int
    p=&number;//stores the address of number variable
    printf("Address of p variable is %u \n",p);
    p=p+1;
    printf("After increment: Address of p variable is %u \n",p); // in our case, p will get incremented by 4 bytes.
    return 0;
}

```

The above code will exhibit different increment values in the pointer when depending upon the type of the pointer.

Addition/subtraction with an integer is similar to increment/decrement, but here the changes in the address contained in the pointer will be a multiple of the size of the data type to which the pointer is pointing.

For e.g. if p is a pointer to an integer type (int *) then p + 5 will lead to p incremented by 5*sizeof(int) bytes. For a platform which provides 4 bytes integer type, p + 4 will increment p by 20 bytes.

If p is a pointer to a double type (double *) then p + 5 will lead to p incremented by 40 bytes.

Difference of two pointers

Two pointers can be subtracted when they point to elements of the same array object. The result of the subtraction is the number of that type that can be accommodated between the two addresses. The following code prints 100 because 100 integers can be accommodated between the addresses pointed by p and q. Taking the difference of two pointers is useful in when accessing array elements with pointers.

```

// source code for difference between two pointers
#include<stdio.h>
int main()
{
    int a[100]; // array of 100 integers
    int * p, *q; // p and q are integer pointers
    p = &a[0] ; // p pointing to the first element of the array
    q = &a[100] ; // q points to the last element of the array (100th )
    printf("\n q-p %u", q-p);
    // prints 100 , because these addresses are separated by 100 integers
} //end of main

```

Comparison of pointers

Two pointers are considered equal if they are pointing to the same memory location.

Types of Pointers

Pointers can be of various types depending upon which data type it is pointing or which object it is pointing. We can have pointers of the following types –

- Pointers to various primitive types such as int , char , float etc.
- Pointers to arrays
- Pointers to pointers
- Pointers to derived types such as structures, unions etc.
- Pointers to functions

- void pointers (generic type , can be typecasted to any type)

In addition to these pointers can also be classified as per the size of the address space in which they are pointing to. According to this pointers can be classified into the following types

- Near pointers (default)
Near pointer is a pointer which is used to bit address of up to 16 bits in a given section of the computer memory that is 16 bit enabled. It can only access data of a small size of about 64 kb in a given period, which is the main disadvantage of this.
- Far pointers
Far pointer is a 32-bit pointer, can access information which is outside the computer memory in a given segment. To use this pointer, one must allocate his/her sector register to store data address in the segment and also another sector register must be stored within the most recent sector.
- Huge pointers
Huge pointer has the same size of 32-bit to that of a far pointer, and it can also access bits that are located outside the sector. Far pointer which is fixed and hence that part of the sector in which they are located cannot be modified in any way; huge pointers can be.

Programs based on Pointers

Q. Write a C program to swap the value of 2 integer variables using pointers.

```
// source code to swap two integer variables using pointers
#include<stdio.h>
int main()
{
    int x=100 , y=200 , temp; // integer variables
    int *p , *q; // pointers to integer type
    p = &x; // p points to x
    q = &y; // q points to y

    //display the values of integers before swapping
    printf("\n x=%d and y=%d ", x , y);
    //swap the two integers using pointers to them
    temp = *p;
    *p = *q;
    *q = temp;
    //display the values of integers after swapping
    printf("\n x=%d and y=%d ", x , y);

} //end of main
```


Introduction to 1-D and 2-D arrays

Arrays are a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is often more useful to think of an array as a collection of variables of the same type.

For scenarios when it is required to work with multiple variables of the same type (like marks of all students of a class in the subject PPC), there are two options

- Make multiple variables of the same type but with different names
- Make an array of that type and access the different variables within the arrays by their index numbers (0,1,2 ..)

The second option is obviously easier and it also allows the programmer to use loops to do operations like reading/writing and manipulating all the variables contained inside the array. Typical use cases for using arrays in a program are

- Storing the marks of 50 students of a class (array of type integer with 50 elements)
- Storing the temperature of a city during the last 30 days (array of type float/double of 30 elements)
- Storing your name maximum length fixed as 20 (array of character with 20 elements)

A 2-D array is actually an array in which every element by itself is an array. Alternatively, it's an array of arrays. 2-D arrays are useful to model 2 dimensional data (like tabular data) in a C program. Typical use cases for using a 2-D array are

- Storing a matrix of 3 rows and 4 columns of real numbers (3*4, 2-D array of float type)
- Storing the board position in a chess game at a given instant (8*8, 2-D array of integers, each storing an integer value representing the chess piece kept on that cell)
- Storing the names of 5 persons, each name of maximum 20 characters length (5*20, 2-D array of characters)

1 D ARRAY:

C	O	D	I	N	G	E	E	K
0	1	2	3	4	5	6	7	8

← single row of elements

2 D ARRAY:

ARRAY:	col 0	col 1	col 2		
		0	1	2	← column
row 0	0	A	A	A	} array elements
row 1	1	B	B	B	
row 2	2	C	C	C	
	↑ rows				

A 1-D array is a single row of elements in which every element is referenced by its index number. For e.g. `arr[3]`

A 2-D array is a collection of M rows and N columns in which every element is referenced by two indices, the row number and the column number. For e.g. `arr[3][2]`

Initialization and Declaration of arrays

Similar to other variables, arrays can also be declared, initialized, read from the console/file and assigned to some values. While some operations can be done directly over an array as a whole, some other can only be done element. For strings (which are essentially null character terminated character arrays) there are some library functions that can perform input/output/assignment operations by taking the string as an argument.

The following table illustrates the difference between working with primitive type variables and array variables.

Operation	Primitive type variables	Arrays
Declaration	int a;	int a[5];
Initialization	int a = 100; char b = 'X' ;	int a[] = { 1,3,5,7,-3}; int b[3] = { 2, 5, 7}; char c[4] = { 'c', 'f', 'x', 'r' };
Assignment of value to variable	a = 20;	Can't directly assign values to the complete array. Assignment can be done in individual array elements a[0] = 100; a[4] = 45; c[2] = 'A';
Copy	a = b;	Can't directly copy one array into another. Individual elements can be copied. Element by element copy can be done (easy with loops). for(i=0; i<10; i++) a[i] = b[i]; // copying elements of array b into a , one at a time
Reading from console	scanf("%d",&a);	Can't directly read the complete array. Can be read element by element (easy with loops). for(i=0; i<5; i++) scanf("%d", arr[i]); // reading element by element for array arr

Character Array and Strings

Character arrays are simple arrays containing characters as its elements. There is no special operation defined over character arrays. Strings are character arrays that are terminated with a special character '\0' (called the Null character). This special property makes it possible for library functions to read/write/copy strings taking it as a whole, like other primitive type variables.

Q. Write a C program to read the elements of an array and print the sum of all elements in it.

```
/* Source code to read the elements of an array and print the sum of all elements of the array */
#include<stdio.h>
int main()
{
    int arr[5]; // array of 5 integers
    int i, sum =0;
    printf("Enter 5 integers ");
    //reading the integers from the console
    for(i=0; i<5; i++)
    {
        printf("\nEnter the %dth integer", i+1); //prompting the user
        scanf("%d", &arr[i]); // reading the i-th element of array
    }
    //calculating the sum of all elements

    for(i=0; i<5; i++)
        sum = sum + arr[i]; //get the cumulative sum of all the elements

    //print the sum
    printf("\nSum of all elements of the array is %d", sum);

} //end of main
```

Q. Write a C program to initialize a 3 * 4 matrix of integers and print the matrix

```
/* Source code to initialize a 3*4 matrix of integers and display it */
#include<stdio.h>
int main()
{
    int mat[3][4] = { {2,4,9, 1} , {1,7,2, 88} , {0,-3,5, -567} };
    int r,c;

    //display the matrix in row - column format
    for(r=0; r<3; r++) //for the 3 rows
    {
        for(c=0; c<4; c++) // for the 4 columns
            printf("%5d", mat[r][c]); // %5d will print in a width of 5
        printf("\n");// line break for each new row of the matrix
    } //end of for r loop ( outer loop)

} //end of main
```

Q. Write a C code to initialize and print a string.

```
/* Source code to initialize and print a string */
#include<stdio.h>
int main()
{
    char str1[] = "This is a simple line of text";
    printf("\nstr = %s", str ); // print the complete string
} //end main
```

Searching – Linear and Binary Search

Searching is the process for detecting the presence/absence of a particular element and on presence finding out the position of the element. Searching is one of the most used operations on data. When you type a query in search engine like google.com it triggers the execution of lot of searching programs at the backend. There are lots of searching algorithms. These algorithms differ in their approach to handle the data and the way to exploit the ordering of data that is being searched. The efficiency of a search algorithm is measured in terms of the number of comparisons performed and the runtime storage requirements of the algorithm.

The two simplest searching algorithms are – Linear search and Binary Search.

Linear Searching

Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.

Linear Search (Array A, Value x)

Step 1: Set i to 1

Step 2: if i > n then go to step 7

Step 3: if A[i] = x then go to step 6

Step 4: Set i to i + 1

Step 5: Go to Step 2

Step 6: Print Element x Found at index i and go to step 8

Step 7: Print element not found

Step 8: Exit

Q. Write a C code to implement Linear Search in C.

```
/* Source code for implmenation of Linear Search in C */
#include<stdio.h>
int main()
{
    int arr[10] = { 2,9,5,7,1,0,6,8,4,3 }; // array of some random integers
    int i, key , FLAG = 0;
    printf("Enter the element to be found( Search key )");
    scanf("%d", &key); // read the search key
    for(i=0; i<10; i++)
    {
        if( arr[i] == key ) // if key is found in some index of the array
        {
            FLAG = 1; // denotes the presense of the key ( Success)
            break ; // no point in continuing with the loop
        }
    }
    //end for

    if(FLAG) // equivalent to if( FLAG == 1)
        printf("%d found at index %d of the array ", key, i);
    else
        printf("%d Not found in the array ", key);
    //end of main
}
```

Binary Search

Binary search is a fast search algorithm as compared to Linear Search. This search algorithm works on the principle of divide and conquers. For this algorithm to work properly, the data collection (generally an array) should be in the sorted form.

Binary search looks for a particular item by comparing the middle most item of the array. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item. Otherwise, the item is searched for in the sub-array to the right of the middle item. This process continues on the sub-array as well until the size of the sub array reduces to zero.

The basic idea is to divide the (array) into 2 halves at each step of the algorithm until either the divided partitions are of size 1 or the element being searched is found to be present at some index of the (array).

Q. Write a C program to implement Binary Search

```
/* Source Code for Iterative Binary Search over an array */
#include<stdio.h>
int main()
{
    int a[10] = {1,3,8,19,25,36,40,41,55,100 }; //sorted array
    int key , left , right , mid , FLAG;
    printf("Enter the element to be searched "); //prompt the user
    scanf("%d", &key); //read the search key
    left = 0; //left index at left bound of the array
    right = 9; // right index at right bound of the array
    while( left <= right) // while the array partition has one or more elements
    {
        mid = (left + right)/2 ; //calculate the mid index
        if(key == a[mid])
        {
            FLAG = 1; // if element is found at mid
            break; // exit out of loop
        }
        else
        if( key < a[mid]) //if key is smaller than mid element
        {
            // look for the key in the left partition
            right = mid-1; // shift right index to mid-1 ,left is unchanged
        }
        else
        {
            // else look for the key in right part ( key > a[mid])
            left = mid + 1;
        }
    }
    //end of while

    if( FLAG) // if element is found at some index
        printf("\n%d found at array index %d ", key, mid);
    else
        printf("\n%d not found in the array ", key);

} //end main
```

Basic Sorting Algorithms – Bubble and Selection Sort

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical (alphabetical) order. The importance of sorting is that data searching can be optimized to a very high level, if data is stored in a sorted manner.

Sorting is also used to represent data in more readable formats. Following are some of the examples of sorting in real-life scenarios –

- **Telephone Directory** – The telephone directory stores the telephone numbers of people sorted by their names, so that the names can be searched easily.
- **Dictionary** – A dictionary stores words in an alphabetical order so that searching of any word becomes easy.

There are various algorithms for sorting. These algorithms differ in their methodology used to sort the data. As a consequence these algorithms need different amount of space (for storing variables in the program) and execution time to sort the data. The most well known sorting algorithms are Bubble Sort , Selection Sort, Quick Sort, Radix Sort,

Insertion Sort , Merge sort etc. The following section describes 2 of the algorithms.

Bubble Sort

The Bubble sort algorithm uses an approach of comparing adjacent elements. If two adjacent elements are found to be out of order then they are swapped. This process is repeated for all pair of adjacent elements until all the possible pairs of adjacent elements are compared. The following algorithm describes this process in an algorithmic manner.

```
begin BubbleSort(list)

  for all elements of list
    if list[i] > list[i+1]
      swap(list[i], list[i+1])
    end if
  end for

  return list

end BubbleSort
```

Working of Bubble Sort

For an array of 5 elements, if every pair of adjacent elements are compared then a total of 10 comparisons. The following image shows the pair of elements being compared at each step of the Bubble sort algorithm. In the following image the elements of the array are being sorted for increasing order. In any comparison of two adjacent elements whenever the right element is found to be smaller than the left element, the elements are swapped.

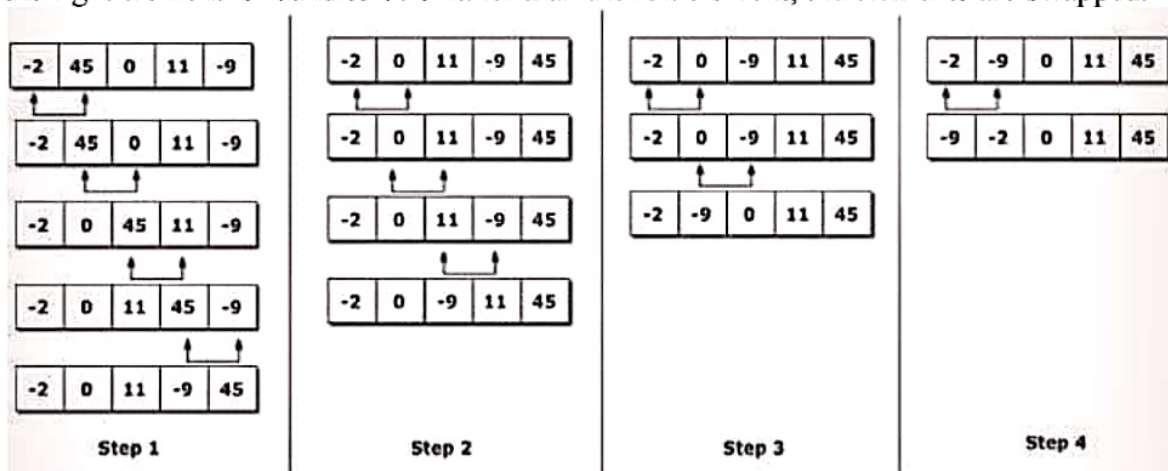


Figure: Working of Bubble sort algorithm

Image Source: <https://fetch-info.blogspot.com/2014/10/bubble-sort.html>

Q. Write a C program to implement Bubble Sort over an array.

```
/* Source code to implement Bubble Sort over an array */
#include<stdio.h>
int main()
{
    int data[5]= {3,4,5,1,2}; // array of 5 integers (random order)
    int i,j,temp;

    //print the array before sorting
    printf("\n Original array is ");
    for(i=0; i<5; i++)
        printf("%d ", data[i]); // print the i-th element

    //sort the array using bubble sort algorithm
    for(i=0; i<4; i++) // this loop executes N-1 times , N is array size
    {
        for(j=0; j<4-i; j++) // j+1 touches upto the last element of the unsorted section
        {
            if( data[j+1] < data[j]) // if the right element is smaller than adjacent left
            {
                //swap the adjacent elements
                temp = data[j];
                data[j] = data[j+1];
                data[j+1]= temp;
            } //end of if
        } //end of inner for ( j loop)

    } //end of outer for ( i loop)

    // print the sorted array
    printf("\nSorted array ");
    for(i=0; i<5; i++)
        printf("%d ", data[i]);

} //end of main
```

```
Original array is 3 4 5 1 2
Sorted array 1 2 3 4 5
```

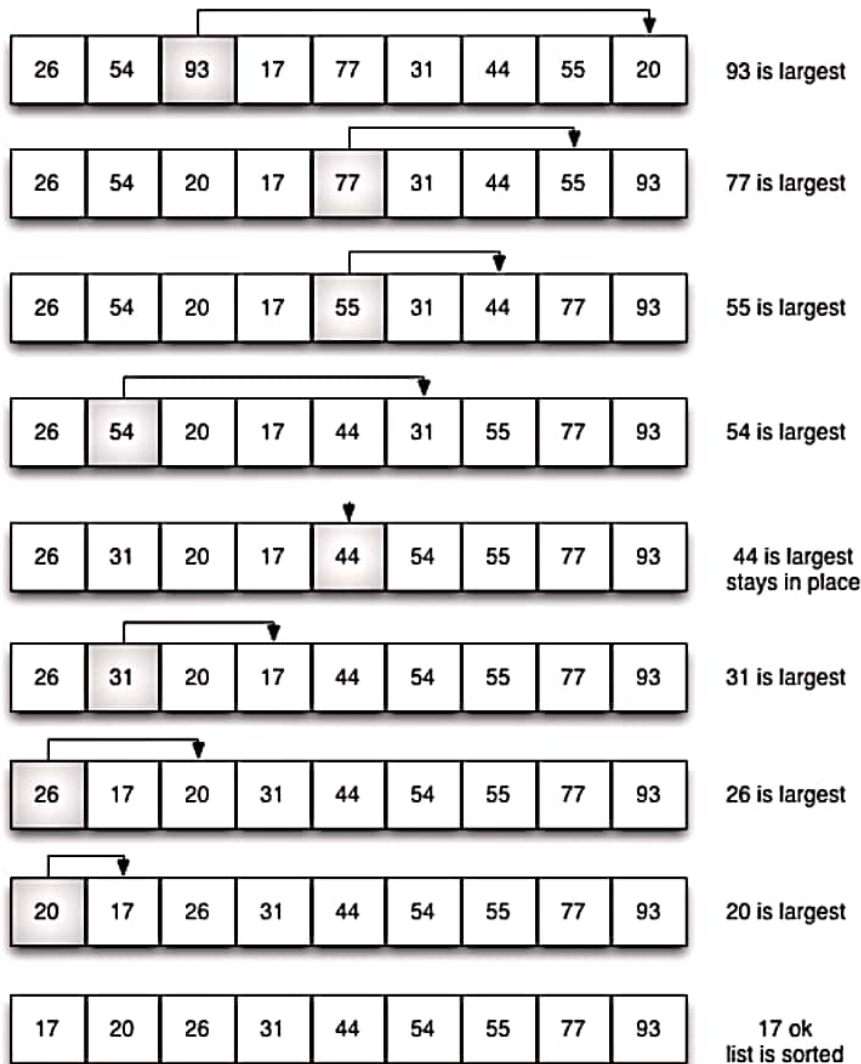
Selection Sort

The Selection sort algorithm uses an approach of “selecting” the appropriate element for the various positions of the array and placing them accordingly. The process of selecting elements requires considering all the elements of the array that have still not been placed in their proper positions.

- Step 1** – Set MIN to location 0
- Step 2** – Search the minimum element in the list
- Step 3** – Swap with value at location MIN
- Step 4** – Increment MIN to point to next element
- Step 5** – Repeat until list is sorted

Working of selection sort

In the following image, Selection sort algorithm has been used to sort the array in ascending order. The algorithm starts selecting the appropriate element for the positions starting from the right most index of the array. Thus at every step one element, out of the remaining elements is being selected and placed in the respective position, according to its rank, starting from the right most position of the array.



<https://runestone.academy/runestone/books/published/pythonds/SortSearch/TheSelectionSort.html>

Implementation of Selection Sort in C

Q. Write a C program to implement Selection Sort over an array.

```
/* Source code to implement Selection Sort over an array */
#include<stdio.h>
int main()
{
    int data[5]= {3,4,5,1,2}; // array of 5 integers (random order)
    int i,j,temp;

    //print the array before sorting
    printf("\n Original array is ");
    for(i=0; i<5; i++)
        printf("%d ", data[i]); // print the i-th element

    //sort the array using selection sort algorithm
    for(i=0; i<4; i++) // this loop executes N-1 times , N is array size
    {
        for(j=i+1; j<5; j++) // j touches upto the last element
        {
            if( data[j] < data[i]) // if the right element is smaller than adjacent left
            {
                //swap the adjacent elements
                temp = data[j];
                data[j] = data[i];
                data[i] = temp;
            } //end of if
        } //end of inner for ( j loop)

    } //end of outer for ( i loop)

    // print the sorted array
    printf("\nSorted array ");
    for(i=0; i<5; i++)
        printf("%d ", data[i]);

} //end of main
```

```
Original array is 3 4 5 1 2
Sorted array 1 2 3 4 5
```

Matrix Addition

Addition of 2 matrices (of the same dimensions) can be done element by element basis, using loops to access the elements according to the loop indices. Two matrices can only be added when they have the same dimension. The following program adds 2 matrices of 3 rows and 4 columns.

Q. Write a code to add two matrices.

```
/* Addition of two matrices */
#include<stdio.h>
int main()
{
    int a[3][4] = { {1,0,2,1}, {4,2,6,1}, {0,-2,4,5} };
    int b[3][4] = { {2,7,-2,0}, {3,2,7,5}, {2,9,-4,1} };
    int c[3][4]; // for storing the sum matrix
    int i,j;
    //addition of two matrices
    for(i=0; i<3; i++) // for the 3 rows
    {
        for(j=0; j<4; j++) // for the 4 columns
            c[i][j] = a[i][j] + b[i][j]; // add the corresponding elements
    }

    //display the sum matrix
    printf("\n Sum matrix is \n");
    for(i=0; i<3; i++)
    {
        for(j=0; j<4; j++)
            printf("%5d",c[i][j] );
        printf("\n");
    }
} //end of main
```

Sum matrix is

3	7	0	1
7	4	13	6
2	7	0	6

Matrix Multiplication

Matrix Multiplication of two matrices can be done in various ways. The simplest way is to have multiply row elements of the first matrix with the corresponding column elements of the second matrix to construct a single element of the product matrix. This process has to be repeated for all the elements of the product matrix. Two matrices can be only being multiplied when they are compatible for matrix multiplication. The compatibility rule requires the number of columns in the first matrix to be equal to the number of rows of the second matrix.

Q. Write a C source code to multiply two matrices.

```
/* Multiplication of two matrices */
#include<stdio.h>
int main()
{
    int a[3][4] = { {1,0,2,1}, {4,2,6,1}, {0,-2,4,5} };
    int b[4][2] = { {0,7}, {0,2}, {0,2}, {0,-1} };
    int c[3][2]; // for storing the product matrix
    int i,j,k;

    //multiplying the matrices
    for(i=0; i<3; i++) // for the 3 rows of the product matrix
    {
        for(j=0; j<2; j++) // for the 2 columns of the product
        {
            c[i][j] = 0; // initialize the product element with 0
            /* compute the product element as sum of element by element
            product of i-th row of a and j-th column of b ( both contain 4 elements each*/
            for(k=0; k<4; k++)
                c[i][j] = c[i][j] + a[i][k] * b[k][j]; // multiply corresponding elements
        } //end of j loop
    } // end of for i loop

    //display the product matrix
    printf("\nProduct matrix is \n");
    for(i=0; i<3; i++)
    {
        for(j=0; j<2; j++)
            printf("%5d",c[i][j] );
        printf("\n");
    }
} //end of main
```

```
Product matrix is
    0    10
    0    43
    0    -1
```

Accessing arrays using Pointers

Elements of any array can be easily accessed using the base address of the array and the offset of the element from the base. The name of the array is the base address of the array. The index number used to access a particular element is used as an offset from the base address. Using a pointer that starts at the base address of the array (name of the array is the base address), one can easily traverse through all the elements of the array. This feature is very useful in many programs like working with arrays inside functions, manipulation of strings etc.

When an array is declared like `int a[5];`

Then the name of the array "a" is the base address. The expression `a[5]` is converted into the base address offset form as

`a[5]` is equivalent to `*(a + 5)` i.e. `a[5]` means the value at the 5th offset address from the base address of the array a. While calculating the offset to be added to the base

address, pointer arithmetic rules are applied. Thus for an int array, to access the element `a[5]`, the offset `5 * sizeof(int)` bytes will be added to the base address of the array. The following code will make the things clearer.

```
/* Accessing array elements using pointers */
#include<stdio.h>
int main()
{
    float arr[3] = { 1.0, 2.5, 25.00 }; // array of floating point type
    int i;
    float *fp ;// pointer to a float type

    printf("\nName of the array is %u", arr); // name is same as base address
    printf("\nBase address of the array is %u", &arr) ; // base address
    printf("\nAddress of 0-th element is %u", &arr[0]) ; //same as base address

    // accessing the array elements using a pointer
    fp = arr; // pointer fp points to the base address now
    for(i=0; i<3; i++)
    {
        printf("\n%d element of array is %f", i, *fp); // value at the address pointed by fp
        fp++; // the pointer moves to the next location of float ( move 4 bytes )
    } //end of for loop
} //end of main
```

```
Name of the array is 3817621988
Base address of the array is 3817621988
Address of 0-th element is 3817621988
0 element of array is 1.000000
1 element of array is 2.500000
2 element of array is 25.000000
```

Standard String Library functions

There are lots of library functions available in the C library for performing common tasks over strings like reading, printing, copying, comparing and counting the length of strings. These functions are declared in the header file `string.h`.

strlen

`size_t strlen(const char *str)`

`size_t` represents unsigned short

It returns the length of the string without including end character (terminating char `'\0'`).

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[20] = "PPS Notes";
    printf("Length of string str1: %d", strlen(str1));
    return 0;
}
```

Strcmp

`int strcmp(const char *str1, const char *str2)`

It compares the two strings and returns an integer value. If both the strings are same (equal) then this function would return 0 otherwise it may return a negative or positive value based on the comparison.

If **string1 < string2** OR **string1 is a substring of string2** then it would result in a negative value. If **string1 > string2** then it would return positive value.

If **string1 == string2** then you would get 0(zero) when you use this function for compare strings.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[20] = "PPS";
    char s2[20] = "pps";
    if (strcmp(s1, s2) == 0)
    {
        printf("string 1 and string 2 are equal");
    } else
    {
        printf("string 1 and 2 are different");
    }
    return 0;
}
```

Strcat

`char *strcat(char *str1, char *str2)`

It concatenates two strings and returns the concatenated string.

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[10] = "Hello";
    char s2[10] = "World";
    strcat(s1, s2);
    printf("Output string after concatenation: %s", s1);
    return 0;
}
```

Strcpy

`char *strcpy(char *str1, char *str2)`

It copies the string str2 into string str1, including the end character (terminator char '\0').

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[30] = "string 1";
    char s2[30] = "string 2 : I'm gonna copied into s1";
    /* this function has copied s2 into s1 */
    strcpy(s1, s2);
    printf("String s1 is: %s", s1);
    return 0;
}
```

DISCLAIMER

The information presented here is merely a collection by the faculty members for their respective teaching assignments. This document does not claim any originality and cannot be used as a substitute for prescribed textbooks. We would like to acknowledge various sources like freely available materials from internet particularly NPTEL course material from which the lecture note was prepared. Further, this document is not intended to be used for commercial purpose and the faculty members are not accountable for any issues, legal or otherwise, arising out of use of this document. The faculty members make no representations or warranties with respect to the accuracy or completeness of the contents of this document.